

Lesson 4

Material

<https://github.com/unixfile/learning-python-2026>

Today

- ▶ if/elif/else
- ▶ Comparison and logical operators
- ▶ for loops and enumerate
- ▶ break and continue
- ▶ while loops
- ▶ List comprehensions
- ▶ Generator expressions

1 if statement

```
password = input("Choose a password: ")
```

```
if password == "12345":  
    print("That's a bad password!")
```

- ▶ Condition is any boolean or truthy value
- ▶ Block runs only when condition is True

2 if/elif/else

```
temperature = int(input("Current temperature: "))

if temperature > 20:
    print("It's warm")
elif temperature > 10:
    print("It's cool")
elif temperature > 5:
    print("It's cold")
else:
    print("It's normal Swedish temperature")
```

- ▶ Only one block runs; the rest are skipped

3 Comparison operators

```
x, y = 5, 10
```

```
x == y    # False - equal
```

```
x != y    # True  - not equal
```

```
x > y     # False - greater than
```

```
x >= y    # False - greater than or equal
```

```
x < y     # True  - less than
```

```
x <= y    # True  - less than or equal
```

4 Logical operators

```
x, y, z = 15, 20, 5
```

```
if x > 10 and y > 10:  
    print("Both greater than 10")
```

```
if x > 20 or z < 10:  
    print("At least one condition true")
```

```
if not (x > y):  
    print("x is not greater than y")
```

5 Pythonic style

Preferred

```
if value is None: ...  
if value is not None: ...  
if my_list: ...  
if not my_list: ...  
if condition: ...
```

Avoid

```
# if value == None: ...  
# if value != None: ...  
# if len(my_list) > 0: ...  
# if len(my_list) == 0: ...  
# if condition == True: ...
```

- ▶ A linter like ruff can flag these automatically

6 for loop

```
fruits = ["banan", "melon", "kiwi", "citron"]
```

```
for fruit in fruits:  
    print(fruit)
```

- ▶ Think of for as “for each”
- ▶ Works on any iterable: lists, strings, files, ...

7 range()

```
In : list(range(5))
```

```
Out: [0, 1, 2, 3, 4]
```

```
In : list(range(2, 8))
```

```
Out: [2, 3, 4, 5, 6, 7]
```

```
In : list(range(0, 10, 2))
```

```
Out: [0, 2, 4, 6, 8]
```

8 break and continue

break exits the loop early

```
for n in range(1, 10):  
    if n == 5:  
        break  
    print(n, end=' ')
```

Output: 1 2 3 4

continue skips to the next iteration

```
for n in range(1, 10):  
    if n % 2 == 0:  
        continue  
    print(n, end=' ')
```

Output: 1 3 5 7 9

9 enumerate

```
fruits = ["banan", "melon", "kiwi", "citron"]  
  
for index, fruit in enumerate(fruits):  
    print(f"{index}: {fruit}")
```

```
# Output:  
# 0: banan  
# 1: melon  
# 2: kiwi  
# 3: citron
```

10 while loop

```
i = 5
while i > 0:
    print(i, end=' ')
    i -= 1
```

Output: 5 4 3 2 1

- ▶ Loop runs as long as the condition is True
- ▶ Forgetting to update i creates an infinite loop

11 while True

```
while True:
    user_input = input("Enter 'exit' to stop: ")
    if user_input == 'exit':
        break
    print(f"You entered: {user_input}")
```

- ▶ break is the only way out of a while True: loop
- ▶ Useful for menus and simple REPLs

12 List comprehensions

```
In : squares = [x**2 for x in range(6)]
```

```
In : squares
```

```
Out: [0, 1, 4, 9, 16, 25]
```

```
In : evens = [x for x in range(10) if x % 2 == 0]
```

```
In : evens
```

```
Out: [0, 2, 4, 6, 8]
```

- Concise syntax for building lists from iterables

13 Generator expressions

```
In : squares = (x**2 for x in range(100_000_000))
```

```
In : type(squares)
```

```
Out: generator
```

- ▶ `()` instead of `[]` — values produced on demand
- ▶ Takes almost no memory
- ▶ Can only be iterated once

```
In : next(squares)
```

```
Out: 0
```

```
In : next(squares)
```

```
Out: 1
```